

Chapter 3

Binary Search Tree

Binary search trees (BSTs) are very simple to understand. We start with a root node with value x , where the left subtree of x contains nodes with values $< x$ and the right subtree contains nodes whose values are $\geq x$. Each node follows the same rules with respect to nodes in their left and right subtrees.

BSTs are of interest because they have operations which are favourably fast: insertion, look up, and deletion can all be done in $O(\log n)$ time. It is important to note that the $O(\log n)$ times for these operations can only be attained if the BST is reasonably balanced; for a tree data structure with self balancing properties see AVL tree defined in §7).

In the following examples you can assume, unless used as a parameter alias that *root* is a reference to the root node of the tree.

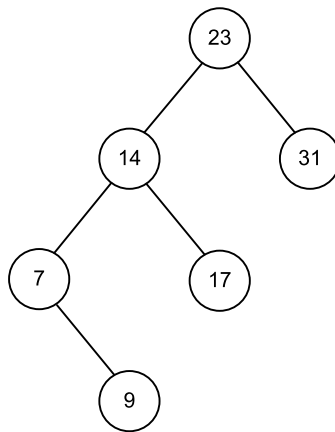


Figure 3.1: Simple unbalanced binary search tree

3.1 Insertion

As mentioned previously insertion is an $O(\log n)$ operation provided that the tree is moderately balanced.

```

1) algorithm Insert(value)
2)   Pre: value has passed custom type checks for type T
3)   Post: value has been placed in the correct location in the tree
4)   if root =  $\emptyset$ 
5)     root  $\leftarrow$  node(value)
6)   else
7)     InsertNode(root, value)
8)   end if
9) end Insert

```

```

1) algorithm InsertNode(current, value)
2)   Pre: current is the node to start from
3)   Post: value has been placed in the correct location in the tree
4)   if value < current.Value
5)     if current.Left =  $\emptyset$ 
6)       current.Left  $\leftarrow$  node(value)
7)     else
8)       InsertNode(current.Left, value)
9)     end if
10)  else
11)    if current.Right =  $\emptyset$ 
12)      current.Right  $\leftarrow$  node(value)
13)    else
14)      InsertNode(current.Right, value)
15)    end if
16)  end if
17) end InsertNode

```

The insertion algorithm is split for a good reason. The first algorithm (non-recursive) checks a very core base case - whether or not the tree is empty. If the tree is empty then we simply create our root node and finish. In all other cases we invoke the recursive *InsertNode* algorithm which simply guides us to the first appropriate place in the tree to put *value*. Note that at each stage we perform a binary chop: we either choose to recurse into the left subtree or the right by comparing the new value with that of the current node. For any totally ordered type, no value can simultaneously satisfy the conditions to place it in both subtrees.

3.2 Searching

Searching a BST is even simpler than insertion. The pseudocode is self-explanatory but we will look briefly at the premise of the algorithm nonetheless.

We have talked previously about insertion, we go either left or right with the right subtree containing values that are $\geq x$ where x is the value of the node we are inserting. When searching the rules are made a little more atomic and at any one time we have four cases to consider:

1. the $root = \emptyset$ in which case $value$ is not in the BST; or
2. $root.Value = value$ in which case $value$ is in the BST; or
3. $value < root.Value$, we must inspect the left subtree of $root$ for $value$; or
4. $value > root.Value$, we must inspect the right subtree of $root$ for $value$.

```
1) algorithm Contains( $root$ ,  $value$ )
2)   Pre:  $root$  is the root node of the tree,  $value$  is what we would like to locate
3)   Post:  $value$  is either located or not
4)   if  $root = \emptyset$ 
5)     return false
6)   end if
7)   if  $root.Value = value$ 
8)     return true
9)   else if  $value < root.Value$ 
10)    return Contains( $root.Left$ ,  $value$ )
11)  else
12)    return Contains( $root.Right$ ,  $value$ )
13)  end if
14) end Contains
```

3.3 Deletion

Removing a node from a BST is fairly straightforward, with four cases to consider:

1. the value to remove is a leaf node; or
2. the value to remove has a right subtree, but no left subtree; or
3. the value to remove has a left subtree, but no right subtree; or
4. the value to remove has both a left and right subtree in which case we promote the largest value in the left subtree.

There is also an implicit fifth case whereby the node to be removed is the only node in the tree. This case is already covered by the first, but should be noted as a possibility nonetheless.

Of course in a BST a value may occur more than once. In such a case the first occurrence of that value in the BST will be removed.

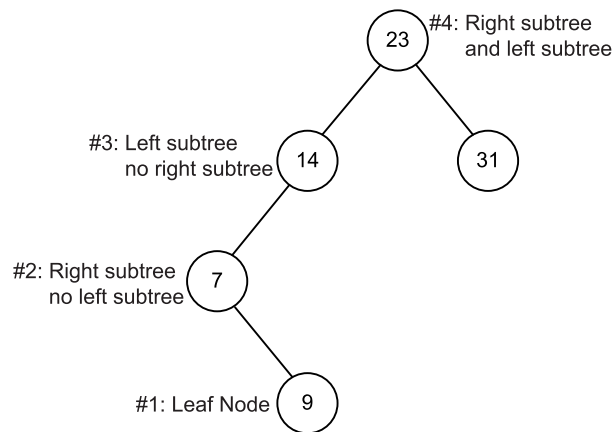


Figure 3.2: binary search tree deletion cases

The *Remove* algorithm given below relies on two further helper algorithms named *FindParent*, and *FindNode* which are described in §3.4 and §3.5 respectively.

```

1) algorithm Remove(value)
2)   Pre: value is the value of the node to remove, root is the root node of the BST
3)   Count is the number of items in the BST
3)   Post: node with value is removed if found in which case yields true, otherwise false
4)   nodeToRemove  $\leftarrow$  FindNode(value)
5)   if nodeToRemove =  $\emptyset$ 
6)     return false // value not in BST
7)   end if
8)   parent  $\leftarrow$  FindParent(value)
9)   if Count = 1
10)    root  $\leftarrow \emptyset$  // we are removing the only node in the BST
11)  else if nodeToRemove.Left =  $\emptyset$  and nodeToRemove.Right = null
12)    // case #1
13)    if nodeToRemove.Value < parent.Value
14)      parent.Left  $\leftarrow \emptyset$ 
15)    else
16)      parent.Right  $\leftarrow \emptyset$ 
17)    end if
18)  else if nodeToRemove.Left =  $\emptyset$  and nodeToRemove.Right  $\neq \emptyset$ 
19)    // case # 2
20)    if nodeToRemove.Value < parent.Value
21)      parent.Left  $\leftarrow$  nodeToRemove.Right
22)    else
23)      parent.Right  $\leftarrow$  nodeToRemove.Right
24)    end if
25)  else if nodeToRemove.Left  $\neq \emptyset$  and nodeToRemove.Right =  $\emptyset$ 
26)    // case #3
27)    if nodeToRemove.Value < parent.Value
28)      parent.Left  $\leftarrow$  nodeToRemove.Left
29)    else
30)      parent.Right  $\leftarrow$  nodeToRemove.Left
31)    end if
32)  else
33)    // case #4
34)    largestValue  $\leftarrow$  nodeToRemove.Left
35)    while largestValue.Right  $\neq \emptyset$ 
36)      // find the largest value in the left subtree of nodeToRemove
37)      largestValue  $\leftarrow$  largestValue.Right
38)    end while
39)    // set the parents' Right pointer of largestValue to  $\emptyset$ 
40)    FindParent(largestValue.Value).Right  $\leftarrow \emptyset$ 
41)    nodeToRemove.Value  $\leftarrow$  largestValue.Value
42)  end if
43)  Count  $\leftarrow$  Count - 1
44)  return true
45) end Remove

```

3.4 Finding the parent of a given node

The purpose of this algorithm is simple - to return a reference (or pointer) to the parent node of the one with the given value. We have found that such an algorithm is very useful, especially when performing extensive tree transformations.

```

1) algorithm FindParent(value, root)
2)   Pre: value is the value of the node we want to find the parent of
3)       root is the root node of the BST and is  $\neq \emptyset$ 
4)   Post: a reference to the parent node of value if found; otherwise  $\emptyset$ 
5)   if value = root.Value
6)     return  $\emptyset$ 
7)   end if
8)   if value < root.Value
9)     if root.Left =  $\emptyset$ 
10)      return  $\emptyset$ 
11)    else if root.Left.Value = value
12)      return root
13)    else
14)      return FindParent(value, root.Left)
15)    end if
16)  else
17)    if root.Right =  $\emptyset$ 
18)      return  $\emptyset$ 
19)    else if root.Right.Value = value
20)      return root
21)    else
22)      return FindParent(value, root.Right)
23)    end if
24)  end if
25) end FindParent

```

A special case in the above algorithm is when the specified value does not exist in the BST, in which case we return $\emptyset$. Callers to this algorithm must take account of this possibility unless they are already certain that a node with the specified value exists.

3.5 Attaining a reference to a node

This algorithm is very similar to §3.4, but instead of returning a reference to the parent of the node with the specified value, it returns a reference to the node itself. Again, $\emptyset$ is returned if the value isn't found.

```

1) algorithm FindNode(root, value)
2)   Pre: value is the value of the node we want to find the parent of
3)   root is the root node of the BST
4)   Post: a reference to the node of value if found; otherwise  $\emptyset$ 
5)   if root =  $\emptyset$ 
6)     return  $\emptyset$ 
7)   end if
8)   if root.Value = value
9)     return root
10)  else if value < root.Value
11)    return FindNode(root.Left, value)
12)  else
13)    return FindNode(root.Right, value)
14)  end if
15) end FindNode

```

Astute readers will have noticed that the *FindNode* algorithm is exactly the same as the *Contains* algorithm (defined in §3.2) with the modification that we are returning a reference to a node not *true* or *false*. Given *FindNode*, the easiest way of implementing *Contains* is to call *FindNode* and compare the return value with $\emptyset$.

3.6 Finding the smallest and largest values in the binary search tree

To find the smallest value in a BST you simply traverse the nodes in the left subtree of the BST always going left upon each encounter with a node, terminating when you find a node with no left subtree. The opposite is the case when finding the largest value in the BST. Both algorithms are incredibly simple, and are listed simply for completeness.

The base case in both *FindMin*, and *FindMax* algorithms is when the Left (*FindMin*), or Right (*FindMax*) node references are $\emptyset$ in which case we have reached the last node.

```

1) algorithm FindMin(root)
2)   Pre: root is the root node of the BST
3)   root  $\neq \emptyset$ 
4)   Post: the smallest value in the BST is located
5)   if root.Left =  $\emptyset$ 
6)     return root.Value
7)   end if
8)   FindMin(root.Left)
9) end FindMin

```

```

1) algorithm FindMax(root)
2)   Pre: root is the root node of the BST
3)   root  $\neq \emptyset$ 
4)   Post: the largest value in the BST is located
5)   if root.Right =  $\emptyset$ 
6)     return root.Value
7)   end if
8)   FindMax(root.Right)
9) end FindMax

```

3.7 Tree Traversals

There are various strategies which can be employed to traverse the items in a tree; the choice of strategy depends on which node visitation order you require. In this section we will touch on the traversals that DSA provides on all data structures that derive from *BinarySearchTree*.

3.7.1 Preorder

When using the preorder algorithm, you visit the root first, then traverse the left subtree and finally traverse the right subtree. An example of preorder traversal is shown in Figure 3.3.

```

1) algorithm Preorder(root)
2)   Pre: root is the root node of the BST
3)   Post: the nodes in the BST have been visited in preorder
4)   if root  $\neq \emptyset$ 
5)     yield root.Value
6)     Preorder(root.Left)
7)     Preorder(root.Right)
8)   end if
9) end Preorder

```

3.7.2 Postorder

This algorithm is very similar to that described in §3.7.1, however the value of the node is yielded after traversing both subtrees. An example of postorder traversal is shown in Figure 3.4.

```

1) algorithm Postorder(root)
2)   Pre: root is the root node of the BST
3)   Post: the nodes in the BST have been visited in postorder
4)   if root  $\neq \emptyset$ 
5)     Postorder(root.Left)
6)     Postorder(root.Right)
7)     yield root.Value
8)   end if
9) end Postorder

```

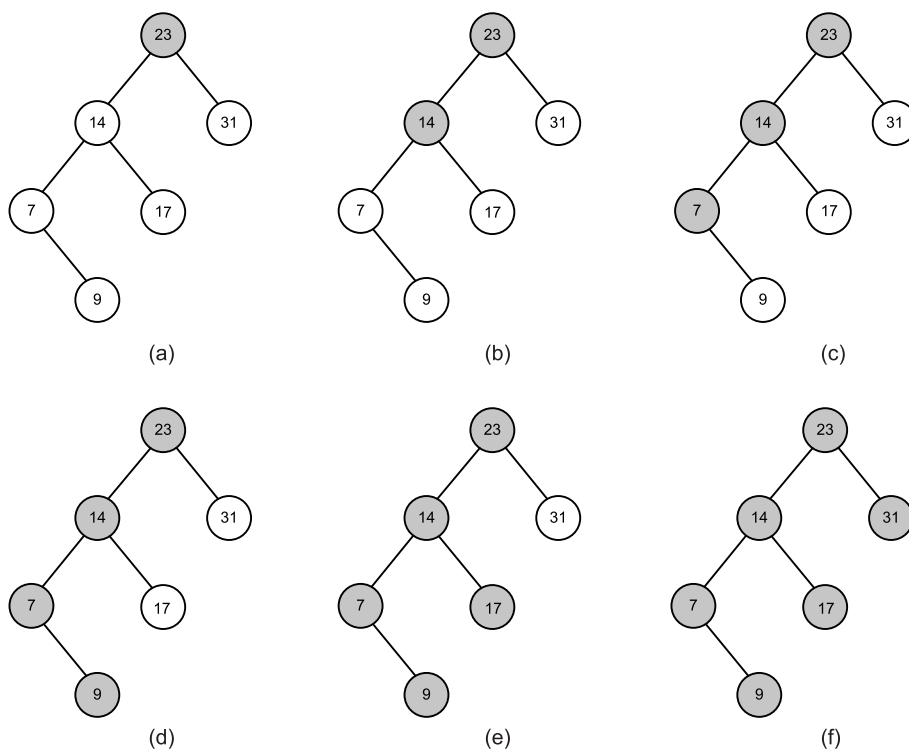



Figure 3.3: Preorder visit binary search tree example

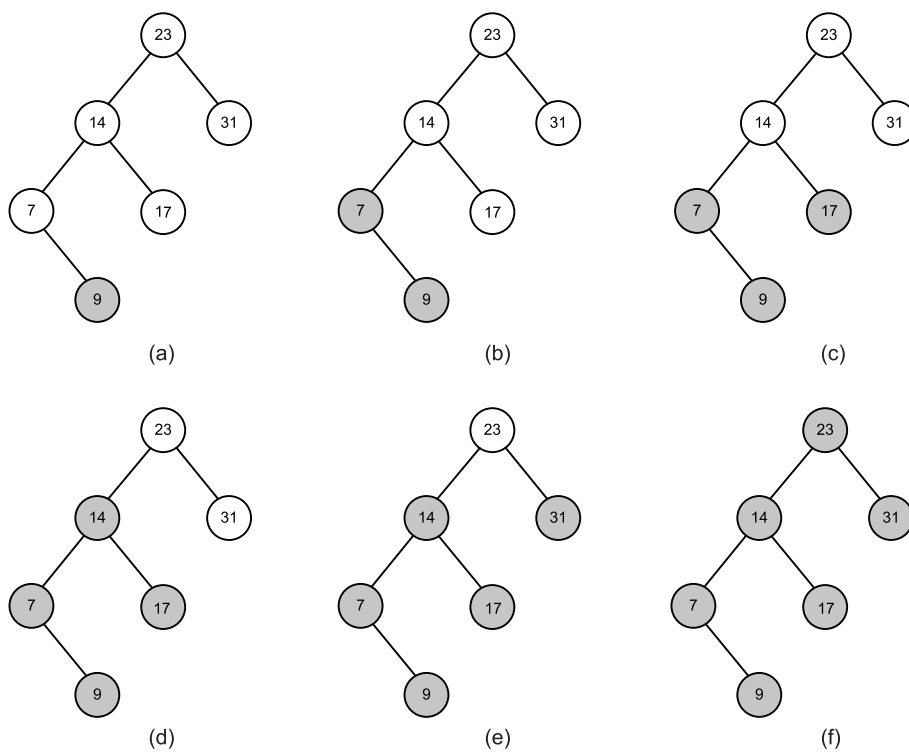


Figure 3.4: Postorder visit binary search tree example

3.7.3 Inorder

Another variation of the algorithms defined in §3.7.1 and §3.7.2 is that of inorder traversal where the value of the current node is yielded in between traversing the left subtree and the right subtree. An example of inorder traversal is shown in Figure 3.5.

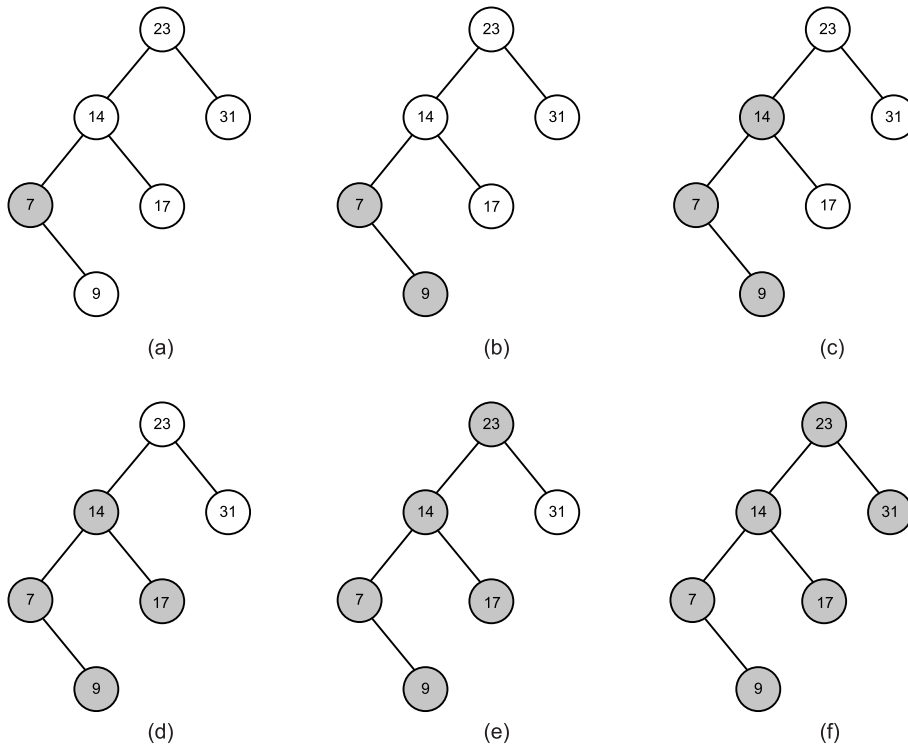


Figure 3.5: Inorder visit binary search tree example

```

1) algorithm Inorder(root)
2)   Pre: root is the root node of the BST
3)   Post: the nodes in the BST have been visited in inorder
4)   if root  $\neq \emptyset$ 
5)     Inorder(root.Left)
6)     yield root.Value
7)     Inorder(root.Right)
8)   end if
9) end Inorder

```

One of the beauties of inorder traversal is that values are yielded in their comparison order. In other words, when traversing a populated BST with the inorder strategy, the yielded sequence would have property $x_i \leq x_{i+1} \forall i$.

3.7.4 Breadth First

Traversing a tree in breadth first order yields the values of all nodes of a particular depth in the tree before any deeper ones. In other words, given a depth d we would visit the values of all nodes at d in a left to right fashion, then we would proceed to $d + 1$ and so on until we had no more nodes to visit. An example of breadth first traversal is shown in Figure 3.6.

Traditionally breadth first traversal is implemented using a list (vector, resizable array, etc) to store the values of the nodes visited in breadth first order and then a queue to store those nodes that have yet to be visited.

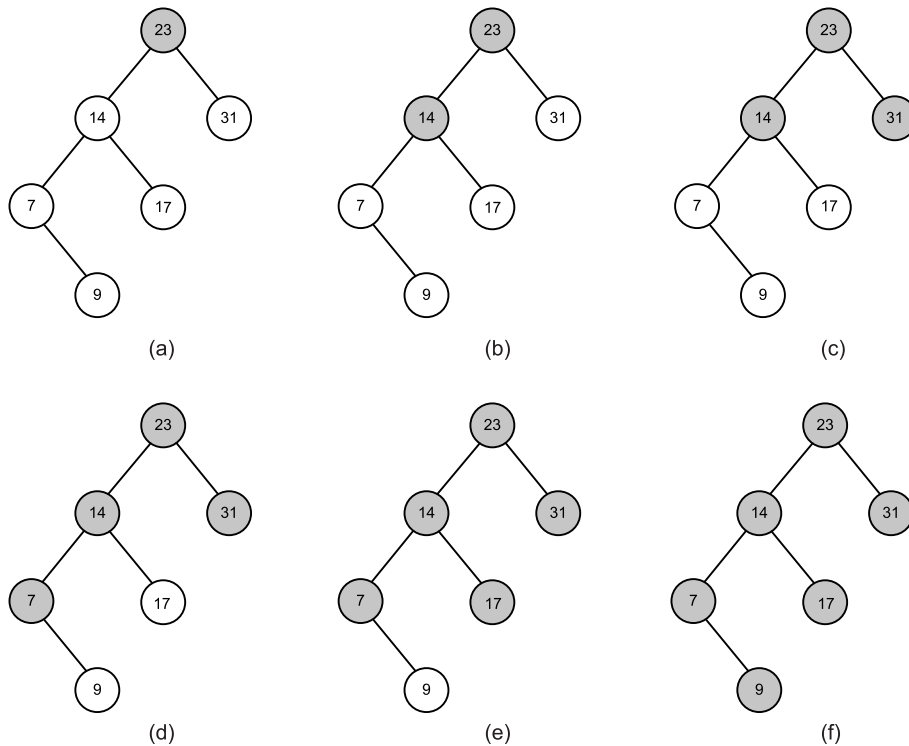


Figure 3.6: Breadth First visit binary search tree example

```

1) algorithm BreadthFirst(root)
2)   Pre: root is the root node of the BST
3)   Post: the nodes in the BST have been visited in breadth first order
4)   q  $\leftarrow$  queue
5)   while root  $\neq \emptyset$ 
6)     yield root.Value
7)     if root.Left  $\neq \emptyset$ 
8)       q.Enqueue(root.Left)
9)     end if
10)    if root.Right  $\neq \emptyset$ 
11)      q.Enqueue(root.Right)
12)    end if
13)    if !q.IsEmpty()
14)      root  $\leftarrow$  q.Dequeue()
15)    else
16)      root  $\leftarrow \emptyset$ 
17)    end if
18)  end while
19) end BreadthFirst

```

3.8 Summary

A binary search tree is a good solution when you need to represent types that are ordered according to some custom rules inherent to that type. With logarithmic insertion, lookup, and deletion it is very efficient. Traversal remains linear, but there are many ways in which you can visit the nodes of a tree. Trees are recursive data structures, so typically you will find that many algorithms that operate on a tree are recursive.

The run times presented in this chapter are based on a pretty big assumption - that the binary search tree's left and right subtrees are reasonably balanced. We can only attain logarithmic run times for the algorithms presented earlier when this is true. A binary search tree does not enforce such a property, and the run times for these operations on a pathologically unbalanced tree become linear: such a tree is effectively just a linked list. Later in §7 we will examine an AVL tree that enforces self-balancing properties to help attain logarithmic run times.

Chapter 4

Heap

A heap can be thought of as a simple tree data structure, however a heap usually employs one of two strategies:

1. min heap; or
2. max heap

Each strategy determines the properties of the tree and its values. If you were to choose the min heap strategy then each parent node would have a value that is $\leq$ than its children. For example, the node at the root of the tree will have the smallest value in the tree. The opposite is true for the max heap strategy. In this book you should assume that a heap employs the min heap strategy unless otherwise stated.

Unlike other tree data structures like the one defined in §3 a heap is generally implemented as an array rather than a series of nodes which each have references to other nodes. The nodes are conceptually the same, however, having at most two children. Figure 4.1 shows how the tree (not a heap data structure) (12 7(3 2) 6(9)) would be represented as an array. The array in Figure 4.1 is a result of simply adding values in a top-to-bottom, left-to-right fashion. Figure 4.2 shows arrows to the direct left and right child of each value in the array.

This chapter is very much centred around the notion of representing a tree as an array and because this property is key to understanding this chapter Figure 4.3 shows a step by step process to represent a tree data structure as an array. In Figure 4.3 you can assume that the default capacity of our array is eight.

Using just an array is often not sufficient as we have to be up front about the size of the array to use for the heap. Often the run time behaviour of a program can be unpredictable when it comes to the size of its internal data structures, so we need to choose a more dynamic data structure that contains the following properties:

1. we can specify an initial size of the array for scenarios where we know the upper storage limit required; and
2. the data structure encapsulates resizing algorithms to grow the array as required at run time

12	7	6	3	2	9
0	1	2	3	4	5

Figure 4.1: Array representation of a simple tree data structure

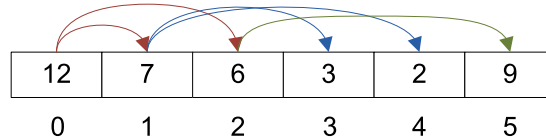


Figure 4.2: Direct children of the nodes in an array representation of a tree data structure

1. Vector
2. ArrayList
3. List

Figure 4.1 does not specify how we would handle adding null references to the heap. This varies from case to case; sometimes null values are prohibited entirely; in other cases we may treat them as being smaller than any non-null value, or indeed greater than any non-null value. You will have to resolve this ambiguity yourself having studied your requirements. For the sake of clarity we will avoid the issue by prohibiting null values.

Because we are using an array we need some way to calculate the index of a parent node, and the children of a node. The required expressions for this are defined as follows for a node at *index*:

1. $(index - 1)/2$ (parent index)
2. $2 * index + 1$ (left child)
3. $2 * index + 2$ (right child)

In Figure 4.4 a) represents the calculation of the right child of 12 ($2 * 0 + 2$); and b) calculates the index of the parent of 3 ($(3 - 1)/2$).

4.1 Insertion

Designing an algorithm for heap insertion is simple, but we must ensure that heap order is preserved after each insertion. Generally this is a post-insertion operation. Inserting a value into the next free slot in an array is simple: we just need to keep track of the next free index in the array as a counter, and increment it after each insertion. Inserting our value into the heap is the first part of the algorithm; the second is validating heap order. In the case of min-heap ordering this requires us to swap the values of a parent and its child if the value of the child is $<$ the value of its parent. We must do this for each subtree containing the value we just inserted.

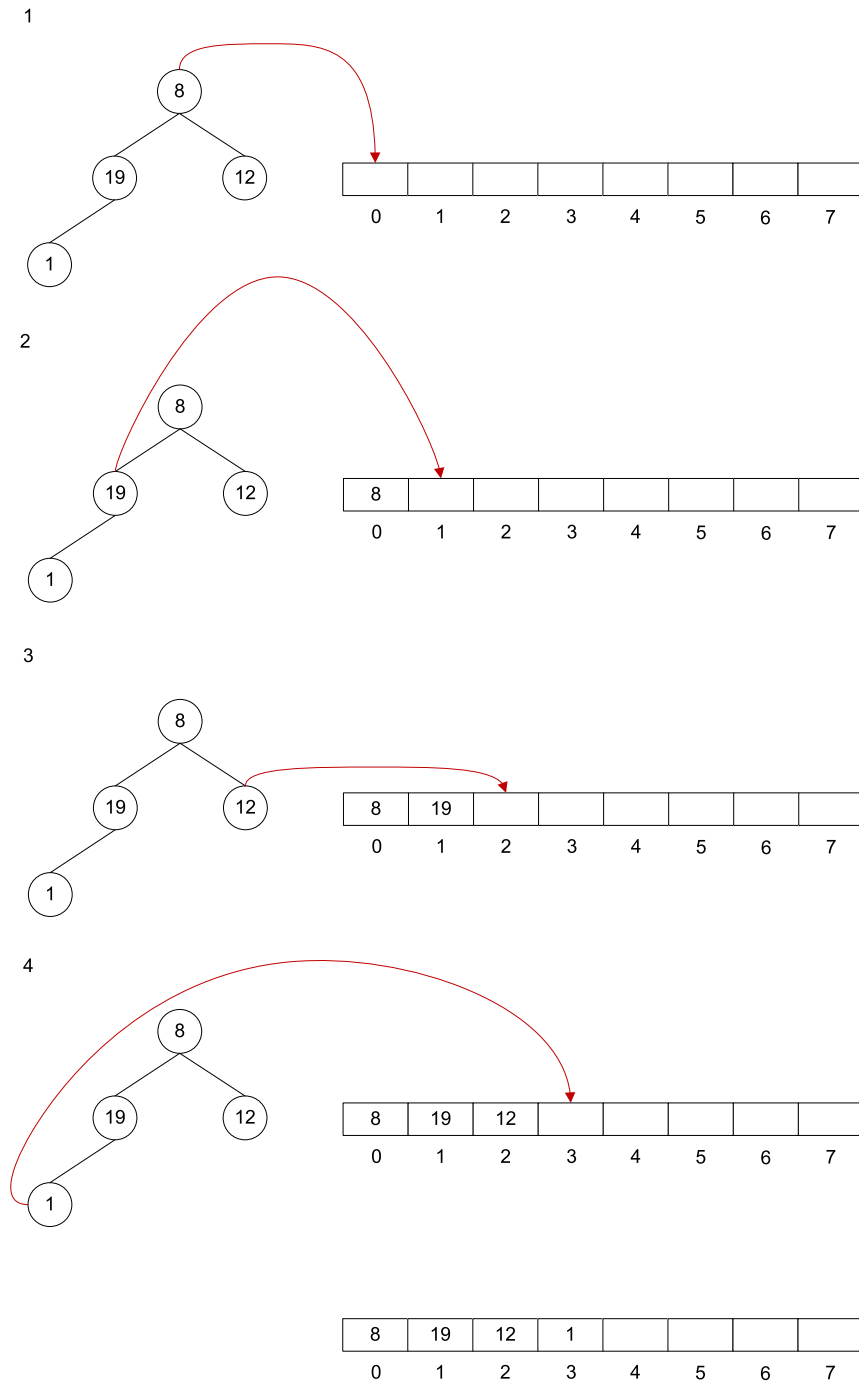


Figure 4.3: Converting a tree data structure to its array counterpart